the source of popular frameworks, in your favorite library, and even in small examples from random blogs.

In fact, metaprogramming is so deeply entrenched in the Ruby language that it's not even sharply separated from "regular" programming. You can't look at a Ruby program and say, "This part here is metaprogramming, while this other part is not." In a sense, metaprogramming is a routine part of every Ruby programmer's job. Once you master it, you'll be able to tap into the full power of the language.

There is also another less obvious reason why you might want to learn metaprogramming. As simple as Ruby looks at first, you can quickly become overwhelmed by its subtleties. Sooner or later, you'll be asking yourself questions such as "Can an object call a `private` method on another object of the same class?" or "How can you define class methods by importing a module?" Ultimately, all of Ruby's seemingly complicated behaviors derive from a few simple rules. Through metaprogramming, you can get an intimate look at the language, learn those rules, and get answers to your nagging questions.

Now that you know what metaprogramming is about, you're ready to dive in.